RecoverAI: Predictive Deleted File Recovery

CARP: Confidence-Adaptive Recovery Prioritization

Major Project External Viva Comprehensive Preparation Guide

Candidate: Koduru Venkata Chandrika (Chandu)
Role: Evaluation & Reporting
Email: koduruvenkatachandrika2006@gmail.com
Affiliation: VIT-AP University (SCOPE)

Chapter 1: Role Overview & Executive Briefing

Role Definition: As the **Evaluation & Reporting Specialist** and Semantic Search Engineer, my core responsibility is transforming raw, cryptic carved files into semantically searchable evidence, formulating our multi-factor retrieval ranking function, evaluating information retrieval metrics (Precision@K), and engineering publication-grade visualization dashboards and automated PDF reports.

Spoken Pitch for External Examiners (30-Second Version):

"Good morning, respected examiners. My name is Koduru Venkata Chandrika, and my individual responsibility is Evaluation, Reporting, and Semantic Search Architecture. When forensic carvers extract files, they dump thousands of documents into flat directories under cryptic names like carved_001.pdf, forcing analysts to manually inspect each file. I designed our dense vector retrieval pipeline using all-MiniLM-L6-v2 embeddings and ChromaDB vector indexing. Furthermore, I authored our Fused Ranking Function that blends semantic similarity with ML confidence and physical block completeness, achieving perfect Precision@1 and Precision@3 across our investigative query benchmarks while generating automated publication-grade evaluation dashboards."

The 1-Minute Technical Summary:

"Traditional keyword searching in digital forensics fails when documents are fragmented, paraphrased, or corrupted. In RecoverAI, I implemented an intelligent evidence indexing engine. First, I built an automated multi-format text extraction pipeline supporting PDF, DOCX, and TXT files. Second, I integrated the all-MiniLM-L6-v2 SentenceTransformer model to project text into a 384-dimensional dense latent space indexed inside ChromaDB using HNSW graphs. Third, recognizing that raw cosine similarity often surfaces corrupted text fragments containing coincidental keyword matches, I formulated the Fused Score: weighting cosine similarity at 60%, ML pre-carve confidence at 25%, and verified block completeness at 15%. This ensured that genuine, intact evidence consistently outranked damaged fragments, delivering Precision@1 = 1.000 across eight natural-language investigative test queries."

Chapter 2: Technical Deep Dive & Mathematical Formulations

1. Dense Vector Semantic Search Architecture:

I implemented the complete evidence indexing and query resolution pipeline in semantic_search/:

- **Multi-Format Extraction (extract.py):** Ingests recovered raw files, determines MIME categories, and extracts clean textual payloads from PDFs (PyPDF), Word documents (python-docx), and plain text files. Fallback logic prevents crashes on partially corrupted headers.
- **Dense Representation (embed.py):** Deploys all-MiniLM-L6-v2 mapping text chunks into a 384-dimensional unit-normalized latent space (||v||_2 = 1.0). Operates 5x faster than BERT-base with sub-millisecond inference per document.
- **Multimodal Extension:** Integrated OpenCLIP (ViT-B-32) for image embedding, enabling cross-modal visual evidence indexing.
- **ChromaDB Vector Store (index.py):** Configured persistent collection recovered_files using Hierarchical Navigable Small World (HNSW) graphs over cosine distance.

2. The Fused Semantic Ranking Function Formula:

In digital forensics, ranking purely by text cosine similarity is dangerous because an overwritten text fragment can coincidentally contain query keywords while being 80% corrupt. I formulated the composite Fused Score:

F(q, d) = 0.60 * CosineSimilarity(q, d) + 0.25 * (P_FULL + P_PARTIAL) + 0.15 * BlockCompleteness

- ***0.60 CosineSimilarity:*** *Ensures semantic query alignment remains the primary retrieval driver.*
- ***0.25 ML Confidence:*** *Rewards documents where Abdul's ML predictor independently confirmed intact cluster structure.*
- ***0.15 BlockCompleteness:*** *Rewards files where Asmin's verification proved non-zero byte payloads.*

3. Information Retrieval Evaluation Metrics (evaluation/metrics.py):

I implemented the evaluation harness measuring search efficacy across eight representative investigative queries:

- **Precision@K:** P@K = (Relevant Documents in Top K) / K.
- **Precision@1 = 1.000:** In 100% of test queries, the exact expected evidentiary document was returned at Rank #1.
- **Precision@3 = 1.000:** Across all queries, the top 3 results contained 100% relevant evidentiary targets.
- **Statistical Metrics:** I also co-authored evaluation/metrics.py calculating Categorical Accuracy (0.8270), Macro AUC-ROC (0.8371), Weighted F1 (0.7915), and False Retirement Rate (15.18%).

4. Automated Reporting & Visualization Engine (docs/):

I authored the automated visualization and reporting scripts: docs/generate_figures.py (generating ROC curves, 3-class confusion matrices, and XGBoost feature importance rankings) and docs/generate_master_results_pdf.py (compiling publication-grade PDF results summaries).

Natural Language Query	Expected File Target	Top Ranked Output	P@1	P@3
recipe for Italian pasta carbonara	cooking_recipe.txt	cooking_recipe.txt	1.0	1.0
UEFA Champions League football match	football_report.txt	football_report.txt	1.0	1.0
corporate tax invoice VAT payment	tax_invoice.pdf	tax_invoice.pdf	1.0	1.0
Python data structures algorithm	python_tutorial.txt	python_tutorial.txt	1.0	1.0
James Webb telescope exoplanet	astronomy_article.txt	astronomy_article.txt	1.0	1.0
penetration testing cybersecurity	cybersecurity_guide.txt	cybersecurity_guide.txt	1.0	1.0
machine learning gradient boosting	ml_research_notes.txt	ml_research_notes.txt	1.0	1.0
Japan travel itinerary Tokyo Kyoto	travel_itinerary.docx	travel_itinerary.docx	1.0	1.0
Mean Evaluation Benchmark Score	**---**	**---**	**1.000**	**1.000**

Chapter 3: Codebase Ownership & Key Functions

• semantic_search/extract.py: Implements extract_text(), handling PDF, DOCX, and TXT decoding with fallback handling.
• semantic_search/embed.py: Implements embed_text() (all-MiniLM-L6-v2), embed_image() (CLIP), and fallback vectors.
• semantic_search/index.py: Manages ChromaDB collections, persistent HNSW indexing, and executes fused_search().
• semantic_search/search_cli.py: Command-line interface for forensic queries.
• evaluation/metrics.py: Implements precision_at_k(), detection_accuracy(), and false_retirement_rate().
• docs/generate_figures.py: Generates ROC curves, confusion matrices, and model comparison bar charts.

Chapter 4: Cross-Module Integration (How My Work Connects)

1. **Integration with Abdul (ML Predictor):** Abdul's ensemble outputs P_FULL and P_PARTIAL probabilities. My Fused Ranking Function takes these values directly as a 25% weight in scoring search candidates.
2. **Integration with Asmin (Carving & Ground Truth):** Asmin's xCarver engine produces the carved files that my extraction pipeline reads. Her SHA-256 block match ratio supplies the BlockCompleteness metric for my fused score.
3. **Integration with Shashwath (Database & QA):** Shashwath maps my ChromaDB document IDs into the search_index table and tests my embedding dimensions in test_semantic.py.

Chapter 5: Top 25 Viva Questions & Defense Answers for Chandu

Q1: What was your individual role in RecoverAI?

I was the Evaluation & Reporting Specialist and Semantic Search Engineer. I built the dense vector search pipeline using all-MiniLM-L6-v2 and ChromaDB, formulated the Fused Ranking Function, evaluated Precision@K metrics, and generated publication-grade visualization dashboards.

Q2: Why is raw cosine similarity dangerous in digital forensics?

In forensics, unallocated sectors often contain fragmented or overwritten text. A corrupt text fragment might coincidentally contain matching keywords and score high on raw cosine similarity. The Fused Score penalizes corrupted fragments by factoring in ML confidence and block completeness.

Q3: Explain your Fused Ranking Function formula.

F(q, d) = 0.60 * CosineSimilarity(q, d) + 0.25 * (P_FULL + P_PARTIAL) + 0.15 * BlockCompleteness. It guarantees that high query relevance only ranks top if the file has verified structural and block integrity.

Q4: Why choose all-MiniLM-L6-v2 instead of BERT-base?

all-MiniLM-L6-v2 generates 384-dimensional embeddings (compared to 768 for BERT-base) and executes 5x faster with minimal semantic loss. In forensic triage on edge laptops, low latency and small memory footprints are critical.

Q5: What was your Precision@1 and Precision@3 score across test queries?

Precision@1 = 1.000 and Precision@3 = 1.000 across all eight representative natural-language test queries.

Q6: How does ChromaDB index vectors?

ChromaDB uses Hierarchical Navigable Small World (HNSW) graphs, an approximate nearest neighbor (ANN) indexing structure that delivers logarithmic search time O(log N) over vector spaces.

Q7: What distance metric did you configure in ChromaDB?

Cosine distance (1 - cosine_similarity). Since our embeddings are L2 unit-normalized, cosine distance is monotonically equivalent to squared Euclidean distance.

Q8: How does your text extractor handle corrupted PDFs?

extract_text() catches parsing exceptions (e.g., EOF marker missing or broken zlib streams) and falls back to filename string embeddings, preventing pipeline crashes on corrupted evidence.

Q9: What happens if ChromaDB is offline or unavailable?

embed.py contains an automated fallback (_fallback_vector) that generates deterministic pseudo-embeddings from SHA-256 byte hashes, allowing unit tests and offline pipelines to function.

Q10: What is Precision@K and how is it calculated?

Precision@K measures the proportion of relevant documents within the top K returned results: P@K = (Relevant in Top K) / min(K, total returned).

Q11: What were the 8 test query categories evaluated in Deliverable C?

Cooking recipe, football match, corporate tax invoice, Python algorithm, astronomy James Webb telescope, cybersecurity penetration testing, machine learning notes, and travel itinerary.

Q12: How does your module integrate with Abdul's work?

I ingest Abdul's predicted recovery confidence (P_FULL + P_PARTIAL) as the 0.25-weighted term in my fused search ranking formula.

Q13: How does your module integrate with Asmin's work?

Asmin provides the carved file artifacts that I parse for text, and her byte-level block match ratio provides the BlockCompleteness metric for my fused score.

Q14: How does your module integrate with Shashwath's work?

Shashwath links my ChromaDB document IDs to the relational search_index table and validates my vector dimensions in test_semantic.py.

Q15: Why did you include OpenCLIP in embed.py?

To support multimodal forensic search, allowing investigators to search recovered JPEG images using natural-language descriptive queries via ViT-B-32.

Q16: How do you normalize embeddings?

Vectors are divided by their Euclidean L2 norm: v_norm = v / np.linalg.norm(v). This ensures dot products directly equal cosine similarities.

Q17: What was the macro AUC-ROC score of the ensemble?

0.8371, demonstrating strong discrimination across all three classes despite heavy class imbalance.

Q18: What was the weighted F1 score of the ensemble?

0.7915 under 5-fold stratified cross-validation.

Q19: What scripts did you use to generate the paper figures?

docs/generate_figures.py generated confusion_matrices_all.png, roc_curves.png, model_comparison.png, and feature_importance.png.

Q20: What did the XGBoost feature importance chart reveal?

Post-deletion filler volume (filler_MB) was the dominant feature (>35% split gain), followed by pre-deletion occupancy (>20%), proving that post-deletion overwriting is the primary threat to file recovery.

Q21: How does your system compare to Amato et al. (2020)?

Amato et al. introduced ontology-based semantic indexing for forensics. RecoverAI advances beyond ontologies by using deep sentence vector embeddings combined with pre-carving ML confidence scoring.

Q22: Why did you evaluate 8 queries instead of random keyword searches?

The 8 queries represent diverse real-world investigative scenarios spanning financial crime, corporate leaks, personal travel, and technical data, rigorously testing semantic synonym matching.

Q23: How fast is query execution in ChromaDB?

Query encoding via all-MiniLM-L6-v2 takes ~12 ms, and HNSW vector traversal takes under 2 ms, delivering total query turnaround under 15 ms.

Q24: What is the benefit of generating automated PDF reports?

In law enforcement and digital forensics, investigative findings must be presented in court-admissible technical summary reports. Our automated generator produces complete, audited PDF evidence summaries.

Q25: What is the single biggest improvement you would make to semantic search?

Implementing hybrid search combining dense vector embeddings with sparse BM25 keyword matching (Reciprocal Rank Fusion), ensuring exact alphanumeric terms like serial numbers or IP addresses are never missed.

Chapter 6: Rapid Revision Cheat Sheet for Chandu

- ***Embedding Model:*** *all-MiniLM-L6-v2 (384-dimensional dense vectors, unit-normalized)*
- ***Vector Database:*** *ChromaDB with persistent HNSW index over cosine distance (semantic_search/index.py)*
- ***Fused Score:*** *F(q,d) = 0.60*Cosine + 0.25*(P_FULL + P_PARTIAL) + 0.15*BlockCompleteness*
- ***Retrieval Benchmark:*** *Precision@1 = 1.000, Precision@3 = 1.000 across 8 investigative test queries*
- ***Reporting Engine:*** *generate_figures.py (ROC curves, confusion matrices) and generate_pdf_report.py*
- ***Key Code:*** *semantic_search/embed.py, extract.py, index.py, evaluation/metrics.py*